

Transient fault handling

All applications that communicate with remote services and resources must be sensitive to transient faults. This is especially true for applications that run in the cloud, where, because of the nature of the environment and connectivity over the internet, this type of fault is likely to be encountered more often. Transient faults include the momentary loss of network connectivity to components and services, the temporary unavailability of a service, and timeouts that occur when a service is busy. These faults are often self-correcting, so, if the action is repeated after a suitable delay, it's likely to succeed.

This article provides general guidance for transient fault handling.

Why do transient faults occur in the cloud?

Transient faults can occur in any environment, on any platform or operating system, and in any kind of application. For solutions that run on local on-premises infrastructure, the performance and availability of the application and its components are typically maintained via expensive and often underused hardware redundancy, and components and resources are located close to each other. This approach makes failure less likely, but transient faults can still occur, as can outages caused by unforeseen events like external power supply or network issues, or other disaster scenarios.

Cloud hosting, including private cloud systems, can offer higher overall availability by using shared resources, redundancy, automatic failover, and dynamic resource allocation across many commodity compute nodes. However, because of the nature of cloud environments, transient faults are more likely to occur. There are several reasons for this:

- Many resources in a cloud environment are shared, and access to these resources is subject to throttling in order to protect the resources. Some services refuse connections when the load rises to a specific level, or when a maximum throughput rate is reached, to allow processing of existing requests and to maintain performance of the service for all users. Throttling helps to maintain the quality of service for neighbors and other tenants that use the shared resource.
- Cloud environments use large numbers of commodity hardware units. They deliver performance by dynamically distributing load across multiple computing units and infrastructure components. They deliver reliability by automatically recycling or replacing failed units. Because of this dynamic nature, transient faults and temporary connection failures might occasionally occur.
- There are often more hardware components, including network infrastructure like routers and load balancers, between the application and the resources and services that it uses.

This additional infrastructure can occasionally introduce additional connection latency and transient connection faults.

- Network conditions between the client and the server might be variable, especially when communication crosses the internet. Even in on-premises locations, heavy traffic loads can slow communication and cause intermittent connection failures.

Challenges

Transient faults can have a big effect on the perceived availability of an application, even if it's been thoroughly tested under all foreseeable circumstances. To ensure that cloud-hosted applications operate reliably, you need to ensure that they can respond to the following challenges:

- The application must be able to detect faults when they occur and determine if the faults are likely to be transient, are long-lasting, or are terminal failures. Different resources are likely to return different responses when a fault occurs, and these responses can also vary depending on the context of the operation. For example, the response for an error when the application is reading from storage might differ from the response for an error when it's writing to storage. Many resources and services have well-documented transient-failure contracts. However, when such information isn't available, it can be difficult to discover the nature of the fault and whether it's likely to be transient.
- The application must be able to retry the operation if it determines that the fault is likely to be transient. It also needs to keep track of the number of times the operation is retried.
- The application must use an appropriate strategy for retries. The strategy specifies the number of times the application should retry, the delay between each attempt, and the actions to take after a failed attempt. The appropriate number of attempts and the delay between each one are often difficult to determine. The strategy varies depending on the type of resource and on the current operating conditions of the resource and the application.

General guidelines

The following guidelines can help you design suitable transient fault handling mechanisms for your applications.

Determine if there's a built-in retry mechanism

- Many services provide an SDK or client library that contains a transient fault handling mechanism. The retry policy it uses is typically tailored to the nature and requirements of the target service. Alternatively, REST interfaces for services might return information that can help you determine whether a retry is appropriate and how long to wait before the next retry attempt.
- You should use the built-in retry mechanism when one is available, unless you have specific and well-understood requirements that make a different retry behavior more appropriate.

Determine if the operation is suitable for retrying

- Perform retry operations only when the faults are transient (typically indicated by the nature of the error) and when there's at least some likelihood that the operation can succeed when retried. There's no point in retrying operations that attempt an invalid operation, like a database update to an item that doesn't exist or a request to a service or resource that suffered a fatal error.
- In general, implement retries only when you can determine the full effect of doing so and when the conditions are well understood and can be validated. Otherwise, let the calling code implement retries. Remember that the errors returned from resources and services outside your control might evolve over time, and you might need to revisit your transient fault detection logic.
- When you create services or components, consider implementing error codes and messages that help clients determine whether they should retry failed operations. In particular, indicate whether the client should retry the operation (perhaps by returning an **isTransient** value) and suggest a suitable delay before the next retry attempt. If you build a web service, consider returning custom errors that are defined within your service contracts. Even though generic clients might not be able to read these errors, they're useful in the creation of custom clients.

Determine an appropriate retry count and interval

- Optimize the retry count and the interval to the type of use case. If you don't retry enough times, the application can't complete the operation and then probably fail. If you retry too many times, or with too short an interval between tries, the application might hold resources like threads, connections, and memory for long periods, which adversely affects the health of the application.
- Adapt values for the time interval and the number of retry attempts to the type of operation. For example, if the operation is part of a user interaction, the interval should

be short and only a few retries should be attempted. By using this approach, you can avoid making users wait for a response, which holds open connections and can reduce availability for other users. If the operation is part of a long running or critical workflow, where canceling and restarting the process is expensive or time-consuming, it's appropriate to wait longer between attempts and retry more times.

- Keep in mind that determining the appropriate intervals between retries is the most difficult part of designing a successful strategy. Typical strategies use the following types of retry interval:
 - **Exponential back-off.** The application waits a short time before the first retry and then exponentially increases the time between each subsequent retry. For example, it might retry the operation after 3 seconds, 12 seconds, 30 seconds, and so on.
 - **Incremental intervals.** The application waits a short time before the first retry, and then incrementally increases the time between each subsequent retry. For example, it might retry the operation after 3 seconds, 7 seconds, 13 seconds, and so on.
 - **Regular intervals.** The application waits for the same period of time between each attempt. For example, it might retry the operation every 3 seconds.
 - **Immediate retry.** Sometimes a transient fault is brief, possibly caused by an event like a network packet collision or a spike in a hardware component. In this case, retrying the operation immediately is appropriate because it might succeed if the fault is cleared in the time that it takes the application to assemble and send the next request. However, there should never be more than one immediate retry attempt. You should switch to alternative strategies, like exponential back-off or fallback actions, if the immediate retry fails.
 - **Randomization.** Any of the retry strategies listed previously can include a randomization to prevent multiple instances of the client sending subsequent retry attempts at the same time. For example, one instance might retry the operation after 3 seconds, 11 seconds, 28 seconds, and so on, while another instance might retry the operation after 4 seconds, 12 seconds, 26 seconds, and so on. Randomization is a useful technique that can be combined with other strategies.
- As a general guideline, use an exponential back-off strategy for background operations, and use immediate or regular interval retry strategies for interactive operations. In both cases, you should choose the delay and the retry count so that the maximum latency for all retry attempts is within the required end-to-end latency requirement.
- Take into account the combination of all factors that contribute to the overall maximum timeout for a retried operation. These factors include the time it takes for a failed

connection to produce a response (typically set by a timeout value in the client), the delay between retry attempts, and the maximum number of retries. The total of all these times can result in long overall operation times, especially when you use an exponential delay strategy where the interval between retries grows rapidly after each failure. If a process must meet a specific service-level agreement (SLA), the overall operation time, including all timeouts and delays, must be within the limits defined in the SLA.

- Don't implement overly aggressive retry strategies. These are strategies that have intervals that are too short or retries that are too frequent. They can have an adverse effect on the target resource or service. These strategies might prevent the resource or service from recovering from its overloaded state, and it continues to block or refuse requests. This scenario results in a vicious circle, where more and more requests are sent to the resource or service. Consequently, its ability to recover is further reduced.
- Take into account the timeout of the operations when you choose retry intervals in order to avoid launching a subsequent attempt immediately (for example, if the timeout period is similar to the retry interval). Also, consider whether you need to keep the total possible period (the timeout plus the retry intervals) below a specific total time. If an operation has an unusually short or long timeout, the timeout might influence how long to wait and how often to retry the operation.
- Use the type of the exception and any data it contains, or the error codes and messages returned from the service, to optimize the number of retries and the interval between them. For example, some exceptions or error codes (like the HTTP code 503, Service Unavailable, with a Retry-After header in the response) might indicate how long the error might last, or that the service failed and won't respond to any subsequent attempt.

Avoid antipatterns

- In most cases, avoid implementations that include duplicated layers of retry code. Avoid designs that include cascading retry mechanisms or that implement retry at every stage of an operation that involves a hierarchy of requests, unless you have specific requirements that require doing so. In these exceptional circumstances, use policies that prevent excessive numbers of retries and delay periods, and make sure you understand the consequences. For example, say one component makes a request to another, which then accesses the target service. If you implement retry with a count of three on both calls, there are nine retry attempts in total against the service. Many services and resources implement a built-in retry mechanism. You should investigate how you can disable or modify these mechanisms if you need to implement retries at a higher level.
- Never implement an endless retry mechanism. Doing so is likely to prevent the resource or service from recovering from overload situations and to cause throttling and refused

connections to continue for a longer time. Use a finite number of retries, or implement a pattern like [Circuit Breaker](#) to allow the service to recover.

- Never perform an immediate retry more than once.
- Avoid using a regular retry interval when you access services and resources on Azure, especially when you have a high number of retry attempts. The best approach in this scenario is an exponential back-off strategy with a circuit-breaking capability.
- Prevent multiple instances of the same client, or multiple instances of different clients, from sending retries simultaneously. If this scenario is likely to occur, introduce randomization into the retry intervals.

Test your retry strategy and implementation

- Fully test your retry strategy under as wide a set of circumstances as possible, especially when both the application and the target resources or services that it uses are under extreme load. To check behavior during testing, you can:
 - Inject transient and nontransient faults into the service. For example, send invalid requests or add code that detects test requests and responds with different types of errors. For examples that use TestApi, see [Fault Injection Testing with TestApi](#) and [Introduction to TestApi – Part 5: Managed Code Fault Injection APIs](#).
 - Create a mockup of the resource or service that returns a range of errors that the real service might return. Cover all the types of errors that your retry strategy is designed to detect.
 - For custom services that you create and deploy, force transient errors to occur by temporarily disabling or overloading the service. (Don't attempt to overload any shared resources or shared services in Azure.)
 - For HTTP-based APIs, consider using a library in your automated tests to change the outcome of HTTP requests, either by adding extra roundtrip times or by changing the response (like the HTTP status code, headers, body, or other factors). Doing so enables deterministic testing of a subset of the failure conditions, for transient faults and other types of failures.
 - Perform high load factor and concurrent tests to ensure that the retry mechanism and strategy works correctly under these conditions. These tests also help ensure that the retry doesn't have an adverse effect on the operation of the client or cause cross-contamination between requests.

Manage retry policy configurations

- A *retry policy* is a combination of all the elements of your retry strategy. It defines the detection mechanism that determines whether a fault is likely to be transient, the type of interval to use (like regular, exponential back-off, and randomization), the actual interval values, and the number of times to retry.
- Implement retries in many places, even in the simplest application, and in every layer of more complex applications. Rather than hard-coding the elements of each policy at multiple locations, consider using a central point to store all policies. For example, store values like the interval and retry count in application configuration files, read them at runtime, and programmatically build the retry policies. Doing so makes it easier to manage the settings and to modify and fine-tune the values in order to respond to changing requirements and scenarios. However, design the system to store the values rather than rereading a configuration file every time, and use suitable defaults if the values can't be obtained from configuration.
- In an Azure Cloud Services application, consider storing the values that are used to build the retry policies at runtime in the service configuration file so that you can change them without needing to restart the application.
- Take advantage of built-in or default retry strategies that are available in the client APIs that you use, but only when they're appropriate for your scenario. These strategies are typically generic. In some scenarios, they might be all you need, but in other scenarios they don't offer the full range of options to suit your specific requirements. To determine the most appropriate values, you need to perform testing to understand how the settings affect your application.

Log and track transient and nontransient faults

- As part of your retry strategy, include exception handling and other instrumentation that logs retry attempts. An occasional transient failure and retry are expected and don't indicate a problem. Regular and increasing numbers of retries, however, are often an indicator of a problem that might cause a failure or that degrades application performance and availability.
- Log transient faults as warning entries rather than as error entries so that monitoring systems don't detect them as application errors that might trigger false alerts.
- Consider storing a value in your log entries that indicates whether retries are caused by throttling in the service or by other types of faults, like connection failures, so that you can differentiate them during analysis of the data. An increase in the number of throttling

errors is often an indicator of a design flaw in the application or the need to switch to a premium service that offers dedicated hardware.

- Consider measuring and logging the overall elapsed times for operations that include a retry mechanism. This metric is a good indicator of the overall effect of transient faults on user response times, process latency, and the efficiency of application use cases. Also log the number of retries that occur so you can understand the factors that contribute to the response time.
- Consider implementing a telemetry and monitoring system that can raise alerts when the number and rate of failures, the average number of retries, or the overall times elapsed before operations succeed is increasing.

Manage operations that continually fail

- Consider how you handle operations that continue to fail at every attempt. Situations like this are inevitable.
 - Although a retry strategy defines the maximum number of times that an operation should be retried, it doesn't prevent the application from repeating the operation again with the same number of retries. For example, if an order processing service fails with a fatal error that puts it out of action permanently, the retry strategy might detect a connection timeout and consider it to be a transient fault. The code retries the operation a specified number of times and then gives up. However, when another customer places an order, the operation is attempted again, even though it then fails every time.
 - To prevent continual retries for operations that continually fail, you should consider implementing the [Circuit Breaker pattern](#). When you use this pattern, if the number of failures within a specified time window exceeds a threshold, requests return to the caller immediately as errors, and there's no attempt to access the failed resource or service.
 - The application can periodically test the service, on an intermittent basis and with long intervals between requests, to detect when it becomes available. An appropriate interval depends on factors like the criticality of the operation and the nature of the service. It might be anything between a few minutes and several hours. When the test succeeds, the application can resume normal operations and pass requests to the newly recovered service.
 - In the meantime, you might be able to fall back to another instance of the service (maybe in a different datacenter or application), use a similar service that offers compatible (maybe simpler) functionality, or perform some alternative operations

based on the hope that the service is available soon. For example, it might be appropriate to store requests for the service in a queue or data store and retry them later. Or you might be able to redirect the user to an alternative instance of the application, degrade the performance of the application but still offer acceptable functionality, or just return a message to the user to indicate that the application isn't currently available.

Other considerations

- When you're deciding on the values for the number of retries and the retry intervals for a policy, consider whether the operation on the service or resource is part of a long-running or multistep operation. It might be difficult or expensive to compensate all the other operational steps that have already succeeded when one fails. In this case, a very long interval and a large number of retries might be acceptable as long as that strategy doesn't block other operations by holding or locking scarce resources.
- Consider whether retrying the same operation could cause inconsistencies in data. If some parts of a multistep process are repeated and the operations aren't idempotent, inconsistencies might occur. For example, if an operation that increments a value is repeated, it produces an invalid result. Repeating an operation that sends a message to a queue might cause an inconsistency in the message consumer if the consumer can't detect duplicate messages. To prevent these scenarios, design each step as an idempotent operation. For more information, see [Idempotency patterns](#).
- Consider the scope of operations that are retried. For example, it might be easier to implement retry code at a level that encompasses several operations and retry them all if one fails. However, doing so might result in idempotency issues or unnecessary rollback operations.
- If you choose a retry scope that encompasses several operations, take into account the total latency of all of them when you determine retry intervals, when you monitor the elapsed times of the operation, and before you raise alerts for failures.
- Consider how your retry strategy might affect neighbors and other tenants in a shared application and when you use shared resources and services. Aggressive retry policies can cause an increasing number of transient faults to occur for these other users and for applications that share the resources and services. Likewise, your application might be affected by the retry policies implemented by other users of the resources and services. For business-critical applications, you might want to use premium services that aren't shared. Doing so provides you with more control over the load and consequent throttling of these resources and services, which can help to justify the extra cost.

Related resources

- [Circuit Breaker pattern](#)
 - [Compensating Transaction pattern](#)
 - [Idempotency patterns](#) 
-

Last updated on 02/24/2023